Sure ✅ Below is a **Spring Boot ↔ Java compatibility matrix** that you can directly use in real projects (production), including **minimum Java**, **max tested Java**, and **what to use today**.

✅ Spring Boot Compatibility Matrix (Java Version Support)

Key rule:
Spring Boot 3.x requires Java 17+ (because Spring Framework 6 uses Java 17 baseline).
(GitHub)

Spring Boot Version Line	Min Java	Max Java (tested/compatible)	Recommended Java for Production	Real-Time Notes
Spring Boot 1.5.x	6	8	Java 8	Very old legacy projects only
Spring Boot 2.0.x	8	9	Java 8	Legacy only
Spring Boot 2.1.x	8	12	Java 8 / 11	Older microservices
Spring Boot 2.2.x – 2.3.x	8	15	Java 11	Common in old enterprise APIs
Spring Boot 2.4.x	8	16	Java 11	Legacy apps
Spring Boot 2.5.x	8	18	Java 11	Stable but old
Spring Boot 2.6.x	8	19	Java 11 / 17	Many companies still run this
Spring Boot 2.7.x	8	21	Java 17	Best option if stuck on Spring Boot 2.x
Spring Boot 3.0.x – 3.2.x	17	21	Java 17 / 21	Major shift to Jakarta EE
Spring Boot 3.3.x	17	23	Java 21	Modern enterprise standard
Spring Boot 3.4.x	17	24	Java 21	Very stable modern platform

Spring Boot 3.5.x – 4.0.x	17	25	Java 21+	Latest line, highest compatibility window
Spring Boot 4.0.1	17	25	Java 21	Current 4.x requires Java 17 baseline (Home)

Sources for Java min/max mapping: (endoflife.date)

✅ Quick Practical Guide (What to Use)

✅ Best Combinations (Real-world Production)

Your Situation	Best Spring Boot	Best Java
New project (2026)	**Spring Boot 3.4 / 3.5 / 4.x**	**Java 21 (LTS)**
Enterprise stable	**Spring Boot 3.2 / 3.3**	**Java 17 or 21**
Legacy but still maintained	**Spring Boot 2.7.x**	**Java 17**
Very old corporate apps	**Spring Boot 2.1–2.5**	**Java 8 / 11**

□ Important Note (Big Breaking Change)

Spring Boot 2.x → 3.x = javax → jakarta migration

That's why Spring Boot 3.x forces Java 17+. (GitHub)

If you want, I can also give you a **Spring Cloud ↔ Spring Boot compatibility matrix** (release trains like 2023.x, 2024.x, 2025.x).